

THE AGENT WHO TAMED THE TURTLE

Learning objectives

Upon completing this lab, you will be able to assign appropriate valences to interactions, enabling a developmental agent to exhibit exploratory behavior in a simulated environment.

Setup

Import the turtle environment

```
In [1]: !pip3 install ColabTurtle
        from ColabTurtle.Turtle import *
```

```
Collecting ColabTurtle
  Downloading ColabTurtle-2.1.0.tar.gz (6.8 kB)
  Preparing metadata (setup.py) ... done
Building wheels for collected packages: ColabTurtle
  Building wheel for ColabTurtle (setup.py) ... done
  Created wheel for ColabTurtle: filename=colabturtle-2.1.0-py3-none-any.whl size=7674 sha256=
bdfecba1ac53cd3dfe870468ffa271e32761f5455307c7b71a3b7014f839b7a3
  Stored in directory: /root/.cache/pip/wheels/8d/30/a2/2b8e815bd3673383f33c0dd80d33ef7651a7c2
57c528515158
Successfully built ColabTurtle
Installing collected packages: ColabTurtle
Successfully installed ColabTurtle-2.1.0
```

Define the Agent class

```
In [2]: class Agent:
        def __init__(self, _valences):
            """ Creating our agent """
            self._valences = _valences
            self._action = None
            self._predicted_outcome = None

        def action(self, _outcome):
            """ tracing the previous cycle """
            if self._action is not None:
                print(f"Action: {self._action}, Prediction: {self._predicted_outcome}, Outcome: {
                    f"Prediction: {self._predicted_outcome == _outcome}, Valence: {self._valence

            """ Computing the next action to enact """
            # TODO: Implement the agent's decision mechanism
            self._action = 0
            # TODO: Implement the agent's anticipation mechanism
            self._predicted_outcome = 0
            return self._action
```

Define the turtle environment class

You don't need to worry about the code of the ColabTurtleEnvironment below.

Just know that this environment:

- interprets the agent's actions as follows `0` : move forward, `1` : turn left, `2` : turn right.
- returns outcome `1` when the turtle bumps into the border of the window, and `0` otherwise.

```
In [3]: # @title Initialize the turtle environment

BORDER_WIDTH = 20

class ColabTurtleEnvironment:

    def __init__(self):
        """ Creating the Turtle window """
        bgcolor("lightGray")
        penup()
        goto(window_width() / 2, window_height()/2)
        face(0)
        pendown()
        color("green")

    def outcome(self, action):
        """ Enacting an action and returning the outcome """
        _outcome = 0
        for i in range(10):
            # _outcome = 0
            if action == 0:
                # move forward
                forward(10)
            elif action == 1:
                # rotate left
                left(4)
                forward(2)
            elif action == 2:
                # rotate right
                right(4)
                forward(2)

            # Bump on screen edge and return outcome 1
            if xcor() < BORDER_WIDTH:
                goto(BORDER_WIDTH, ycor())
                _outcome = 1
            if xcor() > window_width() - BORDER_WIDTH:
                goto(window_width() - BORDER_WIDTH, ycor())
                _outcome = 1
            if ycor() < BORDER_WIDTH:
                goto(xcor(), BORDER_WIDTH)
                _outcome = 1
            if ycor() > window_height() - BORDER_WIDTH:
                goto(xcor(), window_height() -BORDER_WIDTH)
                _outcome = 1

            # Change color
            if _outcome == 0:
                color("green")
            else:
                # Finit l'interaction
                color("red")
                # if action == 0:
                #     break
                if action == 1:
                    for j in range(10):
                        left(4)
```

```

        elif action == 2:
            for j in range(10):
                right(4)

            break

    return _outcome

```

Define the valence of interactions

```

In [4]: valences = [[1, -1],
                    [-1, -1],
                    [-1, -1]]

```

The valence table specifies the valence of each interaction. An interaction is a tuple (action, outcome):

	0 Not bump	1 Bump
0 Forward	1	-1
1 Left	-1	-1
2 Right	-1	-1

Instantiate the agent

```

In [5]: a = Agent(valences)

```

Run the simulation

```

In [6]: # @title Run the simulation

initializeTurtle()

# Parameterize the rendering
bgcolor("lightGray")
penup()
goto(window_width() / 2, window_height()/2)
face(0)
pendown()
color("green")
speed(10)

e = ColabTurtleEnvironment()

outcome = 0
for i in range(10):
    action = a.action(outcome)
    outcome = e.outcome(action)

```

```
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: 1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: 1
Action: 0, Prediction: 0, Outcome: 0, Prediction: True, Valence: 1
Action: 0, Prediction: 0, Outcome: 1, Prediction: False, Valence: -1
Action: 0, Prediction: 0, Outcome: 1, Prediction: False, Valence: -1
Action: 0, Prediction: 0, Outcome: 1, Prediction: False, Valence: -1
Action: 0, Prediction: 0, Outcome: 1, Prediction: False, Valence: -1
Action: 0, Prediction: 0, Outcome: 1, Prediction: False, Valence: -1
Action: 0, Prediction: 0, Outcome: 1, Prediction: False, Valence: -1
```

Observe the turtle moving in a straight line until it bumps into the border of the window

PRELIMINARY EXERCISE

Copy Agent2 that you designed in your previous assignment to this notebook.

Observe how your Agent2 behaves in this environment

ASSIGNMENT

Implement Agent3 by modifying your previous Agent2 such that it can select 3 possible actions: 0, 1, or 2.

Choose the valences of interactions so that the agent does not remain stuck in a corner of the environment.

Create Agent3 by overriding the class Agent or your previous class Agent2

```
In [7]: import random

class Agent3(Agent):
    """ Agent2 généralisé à 3 actions (0: avancer, 1: gauche, 2: droite) """
```

```

def __init__(self, _valences, seuil_ennui=4):
    super().__init__( _valences)
    self.souvenirs = {} # action -> dernier outcome observé
    self.compteur_ennui = 0
    self.seuil_ennui = seuil_ennui

def action(self, _outcome):

    if self._action is not None:
        prediction_correcte = (self._predicted_outcome == _outcome)
        valence_obtenue = self._valences[self._action][_outcome]

        print(f"Action: {self._action}, Prediction: {self._predicted_outcome}, "
              f"Outcome: {_outcome}, Prediction correcte: {prediction_correcte}, "
              f"Valence: {valence_obtenue}")

        self.souvenirs[self._action] = _outcome

        # Un virage réussi prouve que l'obstacle n'est plus devant : on l'oublie
        if self._action != 0 and _outcome == 0 and self.souvenirs.get(0) == 1:
            del self.souvenirs[0]

        if prediction_correcte and valence_obtenue > 0:
            self.compteur_ennui += 1
        else:
            self.compteur_ennui = 0

    if self.compteur_ennui >= self.seuil_ennui:
        # Ennui : une autre action, tirée au hasard parmi les deux autres
        self._action = random.choice([a for a in range(3) if a != self._action])
        self.compteur_ennui = 0
    else:
        positives = {a: self._valences[a][0] for a, o in self.souvenirs.items()
                     if self._valences[a][o] > 0}
        if positives:
            meilleure = max(positives.values())
            meilleures = [a for a, v in positives.items() if v == meilleure]
            # Persistance : on garde l'action courante si elle est parmi les meilleures
            if self._action in meilleures:
                self._action = self._action
            else:
                self._action = random.choice(meilleures)
        else:
            inconnues = [a for a in range(3) if a not in self.souvenirs]
            self._action = random.choice(inconnues or list(range(3)))

    self._predicted_outcome = self.souvenirs.get(self._action, 0)
    return self._action

```

Choose the valence table

Replace the `valences` table by your choice in the code below

```

In [13]: valences = [[1, -1],
                     [-1, -1],
                     [-1, -1]]

```

Test your agent in the TurtleEnvironment

```

In [14]: initializeTurtle()

```

```
bgcolor("lightGray")
penup()
goto(window_width() / 2, window_height()/2)
face(0)
pendown()
color("green")
speed(10)

a = Agent3(valences)
e = ColabTurtleEnvironment()

outcome = 0
for i in range(100):
    action = a.action(outcome)
    outcome = e.outcome(action)
```



[illegible]

```

Action: 0, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 0, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 0, Prediction: 0, Outcome: 1, Prediction correcte: False, Valence: -1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 1, Prediction: 0, Outcome: 1, Prediction correcte: False, Valence: -1
Action: 2, Prediction: 0, Outcome: 1, Prediction correcte: False, Valence: -1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 1, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 1, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 1, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 1, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 1, Prediction: 1, Outcome: 0, Prediction correcte: False, Valence: -1
Action: 0, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 0, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 0, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 0, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 2, Prediction: 1, Outcome: 0, Prediction correcte: False, Valence: -1
Action: 0, Prediction: 0, Outcome: 1, Prediction correcte: False, Valence: -1
Action: 1, Prediction: 0, Outcome: 1, Prediction correcte: False, Valence: -1
Action: 2, Prediction: 0, Outcome: 1, Prediction correcte: False, Valence: -1
Action: 1, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1
Action: 0, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: -1

```

Improve your agent's code

If your agent gets stuck against a border or in a corner, modify the valences or the code. Try different ways to handle boredom or to select random actions. In the next lab, you will see how to design an agent that can adapt to the context.

```

In [15]: valences = [[2, -2],
                    [1, 1],
                    [1, 1]]

```

```

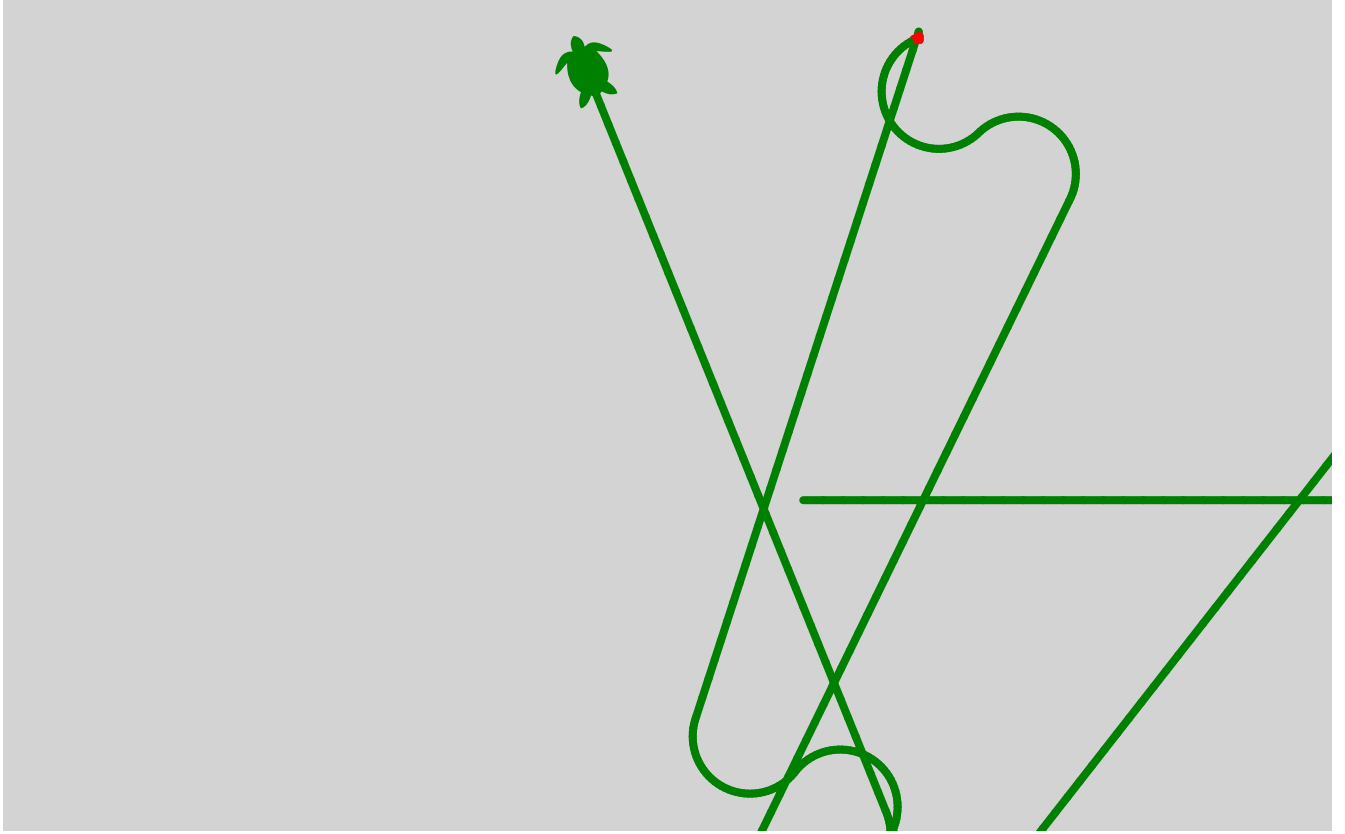
In [16]: initializeTurtle()

bgcolor("lightGray")
penup()
goto(window_width() / 2, window_height()/2)
face(0)
pendown()
color("green")
speed(10)

a = Agent3(valences)
e = ColabTurtleEnvironment()

outcome = 0
for i in range(100):
    action = a.action(outcome)
    outcome = e.outcome(action)

```



[illegible]

```

Action: 1, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 0, Prediction: 0, Outcome: 1, Prediction correcte: False, Valence: -2
Action: 1, Prediction: 0, Outcome: 1, Prediction correcte: False, Valence: 1
Action: 1, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: 1
Action: 1, Prediction: 1, Outcome: 0, Prediction correcte: False, Valence: 1
Action: 1, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 1, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 1, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 0, Prediction: 0, Outcome: 1, Prediction correcte: False, Valence: -2
Action: 2, Prediction: 0, Outcome: 1, Prediction correcte: False, Valence: 1
Action: 2, Prediction: 1, Outcome: 0, Prediction correcte: False, Valence: 1
Action: 2, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 2, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 2, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 0, Prediction: 0, Outcome: 1, Prediction correcte: False, Valence: -2
Action: 2, Prediction: 0, Outcome: 1, Prediction correcte: False, Valence: 1
Action: 2, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: 1
Action: 2, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: 1
Action: 2, Prediction: 1, Outcome: 1, Prediction correcte: True, Valence: 1
Action: 2, Prediction: 1, Outcome: 0, Prediction correcte: False, Valence: 1
Action: 2, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 2, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 2, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 2, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 1, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 1, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 1, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 1
Action: 0, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 2
Action: 0, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 2
Action: 0, Prediction: 0, Outcome: 0, Prediction correcte: True, Valence: 2

```

Report

Explain what you programmed and what results you observed. Export this document as PDF including your code, the traces you obtained, and your explanations below (no more than a few paragraphs):

Implémentation

On part de l'Agent2 (mémoire `action -> outcome` et compteur d'ennui) que l'on généralise à trois actions : 0 avancer, 1 tourner à gauche, 2 tourner à droite. L'agent choisit, parmi les actions dont l'interaction prédite a une valence positive, celle de plus forte valence ; s'il n'en connaît aucune, il explore une action inconnue. Quand l'ennui atteint 4 cycles, il tire au hasard une des deux autres actions (le `1 - action` de l'Agent2 ne fonctionne plus à trois actions).

Valences

On choisit `[[2, -2], [1, 1], [1, 1]]` : avancer sans bump est l'interaction préférée, se cogner est désagréable, et tourner est toujours agréable, même contre le bord. Avec la table par défaut `[[1, -1], [-1, -1], [-1, -1]]`, toutes les interactions sont négatives dès que l'agent touche le bord. Il n'a alors plus d'action positive à choisir, tire au hasard et reste collé au mur (plus de 60 cycles de bump consécutifs dans la première trace).

Modifications du code

On ajoute deux modifications simples contre le coincement : (1) la persistance, l'agent garde son virage tant qu'il est le meilleur choix (sinon gauche et droite alternent au hasard et s'annulent contre le mur) ; (2) après un virage sans bump, il oublie que « avancer = bump », car la mémoire ne tient pas compte du contexte et resterait sinon bloquée sur une prédiction périmée.

Résultats observés

Simulation sans interface. On reproduit la géométrie de l'environnement (10 graines, 300 cycles) pour comparer plusieurs tables de valences. Avec `[[2, -2], [1, 1], [1, 1]]`, la série de bumps consécutifs la plus longue est de 7 cycles, contre 63 à 84 cycles avec `[[1, -1], [-1, -1], [-1, -1]]` et `[[2, -2], [1, -1], [1, -1]]`.

Dans TurtlePy. Sur 100 cycles avec `[[2, -2], [1, 1], [1, 1]]`, l'agent n'a heurté le bord qu'en avançant 5 fois (valence -2). Après chaque choc, il tourne pendant 2 à 7 cycles puis repart vers l'intérieur. La série de bumps consécutifs la plus longue est de 7 cycles, comme dans la simulation. Il alterne avancées (valence 2) et virages (valence 1) par séquences de 4 cycles, ce qui correspond au seuil d'ennui, et ne reste pas coincé dans un coin. Les erreurs de prédiction apparaissent surtout au contact du bord.

Limite

La mémoire associe une seule prédiction à chaque action, sans tenir compte du contexte (la position de la tortue). L'agent ne peut donc pas anticiper qu'avancer cogne près d'un bord et pas au centre : ce sera l'objet de l'Agent4.

In []: